

TECHNICAL ANALYSIS REPORT: INTEGRATED IOT & EDGE-AI ACOUSTIC EARLY WARNING SYSTEM FOR HUMAN-ELEPHANT CONFLICT MITIGATION

NOTE

****[!NOTE]****

> This report is a read-only technical audit of the existing codebase. No files were modified, created in the project repository, or deleted during this analysis.

1. PROJECT STRUCTURE

```
Complete System/
├── Integrated-IoT-and-Edge-AI-Acoustic-Early-Warning-System-for-Human-Elephant-Conflict-Mitigation-main/
│   ├── .env # Root environment variables (Supabase, VAPID, Flask secret)
│   ├── .gitignore # Git ignore rules
│   ├── ALERT_SYSTEM_README.md # Alert & Notification subsystem documentation
│   ├── WEB_PUSH_README.md # Web Push notification guide
│   ├── alert_config.json # Cooldown and threshold configuration
│   ├── alert_history.json # Local JSON fallback history
│   ├── api/
│   │   └── index.py # Vercel serverless entry point wrapper
│   ├── camera_alert_monitor.py # Standalone daemon for continuous camera polling
│   ├── create_env_v2.py # Utility to generate/update .env file
│   ├── fix_all_timestamps.py # Timestamp normalization helper script
│   ├── generate_vapid.py # Script to generate VAPID keypair for Web Push
│   ├── generate_vapid_simple.py # Simplified VAPID key generator
│   ├── hardware_integration/
│   │   ├── esp32_node.ino # Core ESP32 Node 1 Arduino sketch (PIR, Sound, GPS, Relay, LCD)
│   │   ├── esp32_node_02.ino # ESP32 Node 2 variant sketch
│   │   ├── esp32_node_03.ino # ESP32 Node 3 variant sketch
│   │   └── esp32_node_04.ino # ESP32 Node 4 variant sketch
│   ├── migrate_production_schema.py # Python database migration script
│   ├── migrate_to_supabase.py # CSV to Supabase data migrator
│   ├── migration_v1_production.sql # SQL DDL for Supabase (Tables, Indexes, RLS, Triggers)
│   ├── notification_service.py # Two-stage alert state machine service
│   ├── police_incidents.json # Local JSON store for police incident dispatch statuses
│   ├── push_service.py # VAPID Web Push notification delivery service
│   ├── push_subscriptions.json # Persistent storage for browser push subscriptions
│   ├── requirements.txt # Python dependencies manifest
│   ├── schema_profiles.sql # SQL schema for user profiles and auth triggers
│   ├── scratch/ # Temporary developer scratch space
│   ├── server.py # Main Flask backend application & REST API engine
│   ├── service-worker.js # Web Push Service Worker for browser notifications
│   ├── static/
│   │   ├── css/
│   │   │   ├── alerts.css # Alert management styling
│   │   │   ├── camera_detection.css # Camera AI live preview & modal styling
│   │   │   ├── landing_and_auth.css # Landing page & authentication UI styling
│   │   │   ├── main.css # Global design system & theme variables
│   │   │   ├── role_dashboards.css # Role-specific dashboard views styling
│   │   │   └── sensor_monitoring.css # Telemetry charts & node card styling
│   │   └── js/
│   │       ├── alerts.js # Alert history & trigger frontend logic
│   │       ├── analytics.js # Analytics tab helper placeholder
│   │       ├── app.js # App initialization, tab switching & header updates
│   │       ├── camera_detection.js # Live camera stream, detection history & test popup UI
│   │       ├── dss.js # Decision Support System UI logic
│   │       ├── edge_ai.js # Acoustic Edge-AI simulation UI logic
│   │       ├── elephant_popup.js # Emergency modal popups for fresh detections
│   │       ├── map.js # Leaflet GIS map, corridor zones & live node markers
│   │       ├── push_subscription.js # Web Push VAPID subscription manager
│   │       ├── sensor_monitoring.js # Chart.js live telemetry charts & node filters
│   │       ├── sensor_nodes.js # Sensor node state management
│   │       ├── service-worker.js # Static service worker copy
│   │       └── user_management.js # Admin user provisioning & role toggle UI logic
│   ├── supabase_client.py # Supabase Python SDK client setup & helper getters
│   ├── sync_local_to_supabase.py # Local CSV/image synchronizer to Supabase Cloud
│   ├── templates/
│   │   ├── 403.html # Access Denied / HTTP 403 page
│   │   ├── index.html # Main Forest Officer Dashboard & Admin Portal
│   │   └── landing.html # Public landing page with role login cards
```

```

├── police_dashboard.html      # Police Incident Response & Patrol Dispatch Dashboard
├── villager_dashboard.html   # Villager Safety & Warning Dashboard
├── test_user_management.py    # Unit/integration test for User Management APIs
├── vercel.json               # Vercel deployment configuration
├── whatsapp_service.py       # Local Baileys WhatsApp integration service
├── WelcomeScreen/
├──   └── fall_final/
├──     ├── 123.py             # Temporary script
├──     ├── coco.data          # COCO class names list (80 classes including 'elephant')
├──     ├── detected_elephants/ # Local directory for saved elephant detection images (.jpg)
├──     ├── elephant_detection_dataset.csv # Local CSV dataset logging all camera frames
├──     ├── frozen_inference_graph.pb # MobileNet SSD v3 Large pre-trained weights binary
├──     ├── human_fall_detect_final_count.py # Primary OpenCV MobileNet SSD camera detection script
├──     ├── human_fall_detect_final_count.md # Documentation for camera detection script
├──     ├── make_docker.py     # Docker creation helper placeholder
├──     └── ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt.txt # MobileNet SSD network architecture config
├── whatsapp-service/
├──   ├── .env                # WhatsApp port and authorized recipient config
├──   ├── .env.example        # Example environment file
├──   ├── auth/               # Legacy auth directory
├──   ├── package.json         # Node.js dependencies (@whiskeysockets/baileys, express, cors)
├──   ├── package-lock.json    # Locked NPM dependency tree
├──   ├── server.js           # Express.js Baileys WhatsApp microservice server (port 3001)
├──   └── sessions/           # Persistent Baileys WhatsApp multi-file auth credentials

```

2. TECHNOLOGY STACK

Frontend

- **Framework:** Vanilla HTML5 / JavaScript (ES6 Modules & dynamic DOM manipulation). Server-side templating powered by Jinja2 (`Flask.render_template`).
- **Language:** JavaScript (Browser ES6) / HTML5.
- **Build Tool:** None (No Vite, Webpack, or Next.js. Served directly as static files by Flask).
- **CSS / UI Framework:** Vanilla Custom CSS Design System (`main.css` , `landing_and_auth.css` , etc.) using modern CSS variables, glassmorphism aesthetics, flexbox/grid. Typography: Google Fonts (Inter, Outfit, Fira Code), FontAwesome 6.5.1 icons.
- **Libraries:**
 - Leaflet.js v1.9.4 (Interactive GIS Corridor Map & GPS node plotting).
 - Chart.js (Real-time sensor telemetry time-series charts).
 - `@supabase/supabase-js` v2 (Client-side Supabase Realtime subscriptions).
- **Routing:** Flask server-side URL routing (`/landing` , `/officer` , `/villager` , `/police`) + client-side tab switcher (`switchTab()`).

Backend

- **Framework:** Flask 3.x (Python).
- **Language:** Python 3.10+.
- **APIs:** RESTful JSON APIs endpoints built into Flask `server.py` .
- **Server:** Werkzeug WSGI (Development), Gunicorn (Production configuration via `requirements.txt`). Secondary Node.js microservice (`whatsapp-service/server.js`) running Express on Port 3001.

AI / ML

- **Model:** MobileNet SSD v3 Large (`ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt.txt` architecture + `frozen_inference_graph.pb` weights).
- **Framework:** OpenCV Deep Neural Network module (`cv2.dnn_DetectionModel`).
- **Computer Vision Libraries:** OpenCV (`cv2`), `numpy` .
- **Detection Pipeline:**
 1. Input webcam capture (1280x720).
 2. Frame resizing to 320x320 with mean subtraction (`127.5, 127.5, 127.5`) and scaling factor `1.0 / 127.5` .
 3. Class filtering against `coco.data` for `object_name == "elephant"` .
 4. Non-Maximum Suppression (NMS threshold = 0.4, confidence threshold = 0.45).
 5. Bounding box drawing, confidence logging, local JPEG frame save, CSV appending, pyttsx3 audio voice alert, and async HTTP POST/Supabase cloud push.

IoT

- **Microcontroller:** ESP32 (DevKit / WROOM).
- **Sensors:**
 - PIR Motion Sensor (Digital Input on Pin D19).
 - Acoustic Sound Sensor DO (Digital Input on Pin D18).
 - GPS Module (UART2: RX=Pin D16, TX=Pin D17) running `TinyGPSPlus` .
 - LCD 16x2 I2C Display (Address `0x27` , SDA=Pin D25, SCL=Pin D26).
- **Actuators:**
 - Strobe Light / Siren Relay (Digital Output on Pin D23, active LOW).
 - Local Status Indicator LED (Digital Output on Pin D22).
- **Communication Method:** Wi-Fi (WPA2 Station mode + fallback AP configuration portal `ESP32_Config` at `192.168.4.1`). HTTP POST JSON telemetry to Flask server every 2000 ms. Local HTTP server on Port 80 for manual strobe commands (`/actuator`).

Database & Storage

- **Database:** Supabase PostgreSQL Cloud.
- **Supabase Usage:** `supabase-py` Client SDK for table queries/inserts/updates and authentication. Supabase JS Client on frontend for Realtime subscriptions.
- **Tables:** `devices` , `detections` , `raw_sensor_data` , `alerts` , `profiles` , `system_settings` , `notification_config` , `notification_history` .
- **Storage Buckets:** `elephant-images` (stores camera snapshot images of detected elephants).

Notifications

- **Current Active Provider:** Local Baileys WhatsApp Microservice (`@whiskeysockets/baileys` Node.js server on `http://localhost:3001`).
- **Email:** NOT FOUND / NOT CLEAR FROM PROJECT (Legacy stubs in `system_settings` , no active SMTP provider implemented).
- **SMS:** NOT FOUND / NOT CLEAR FROM PROJECT (Legacy references to Twilio in `requirements.txt` , but replaced by local WhatsApp microservice).
- **WhatsApp:** ACTIVE (Sends formatted alert messages with images directly to authorized recipient `+917094528671`).
- **Web Notifications:** ACTIVE (VAPID Web Push delivered via `pywebpush` and browser `service-worker.js`).

3. APPLICATION FLOW



1. **Trigger:** Physical movement (PIR) or loud infrasonic/acoustic frequency (Sound sensor) triggers ESP32.

2. **Sensor Data Destination:** Transmitted via HTTP POST to Flask `/api/esp32/data` or `/api/node-event`. Stored immediately in Supabase table `raw_sensor_data`.

3. **Camera & AI Processing:** When `server.py` receives telemetry with `PIR` or `Sound = true`, `trigger_human_fall_detection()` launches `WelcomeScreen/fall_final/human_fall_detect_final_count.py` in a separate OS console window (with a 15s debounce cooldown).
4. **Elephant Confirmation:** OpenCV MobileNet SSD detects object `elephant` with confidence $\geq 45\%$.
5. **Detection Storage:** Saved locally in `WelcomeScreen/fall_final/detected_elephants/` and `elephant_detection_dataset.csv`. Then uploaded to Supabase Storage bucket `elephant-images` and inserted into Supabase `detections` table (`detection="ELEPHANT_DETECTED"`).
6. **Dashboard Data Reception:** Frontend JS subscribes to Supabase Realtime changes on `detections` table and polls `/api/status`, `/api/nodes`, and `/api/elephant-detection/confirmed` every 2-3 seconds.
7. **Notification Trigger:** Upon DB insert, `sync_detection_record()` calls `whatsapp_service.send_confirmed_elephant_whatsapp()` which sends a POST request to `http://localhost:3001/send-image` (or `/send-message`) targeting `+917094528671`. Concurrently, `push_service.py` dispatches Web Push notifications.

4. ESP32 / IOT ANALYSIS

- **Board Type:** ESP32 DevModule / WROOM-32.
- **Device ID:** ESP32-NODE-01 (hardcoded in `esp32_node.ino`), ESP32-NODE-02 (in `esp32_node_02.ino`).
- **Wi-Fi Configuration:** Connects to saved Wi-Fi credentials via `Preferences.h`. If connection fails, launches configuration AP `ESP32_Config` (password `12345678`) at `192.168.4.1`.
- **Pin Definitions:**
 - `PIR_PIN` : 19 (Digital Input)
 - `SOUND_PIN` : 18 (Digital Input, `SOUND_ACTIVE_STATE = HIGH`)
 - `RELAY_PIN` : 23 (Digital Output, Active LOW: `RELAY_ON = LOW`, `RELAY_OFF = HIGH`)
 - `LED_PIN` : 22 (Digital Output, Local status indicator)
 - `GPS_RX_PIN` : 16 (HardwareSerial 2 RX)
 - `GPS_TX_PIN` : 17 (HardwareSerial 2 TX)
 - `LCD_SDA_PIN` : 25 (I2C SDA)
 - `LCD_SCL_PIN` : 26 (I2C SCL)
- **Baud Rates:** Serial Debug = `115200` baud; GPS HardwareSerial(2) = `9600` baud (default TinyGPSPlus).
- **Serial Ports:** Serial0 (USB Debug), Serial2 (GPS Module).
- **API Endpoints Called:** HTTP POST `http://10.238.159.122:5000/api/esp32/data` (configured via `SERVER_URL`).
- **Payload Structure Sent by ESP32:**

```
`json
{
  "deviceId": "ESP32-NODE-01",
  "pir": true,
  "sound": false,
  "gpsFix": true,
  "latitude": 12.822400,
  "longitude": 77.577000,
  "satellites": 8,
  "alert": true,
```

```
"wifiRSSI": -68,
"ledStrobe": false,
"strobeManualControl": false
}
```

- **Data Transmission Frequency:** Every 2000 ms (`SEND_INTERVAL = 2000`).
- **Sensor Trigger Logic:**
- **PIR Detected:** Sets `pirDetected = true` . Starts 5-second alert timer (`ALERT_DURATION = 5000`). LCD shows "PIR DETECTED" . Telemetry payload sends `"pir": true` .
- **Sound Detected:** Sets `soundDetected = true` . Starts 5-second alert timer. LCD shows "SOUND DETECTED" . Telemetry payload sends `"sound": true` .
- **Both Detected:** Sets both flags true. LCD shows "PIR + SOUND" . Telemetry payload sends `"pir": true, "sound": true` .
- **Neither Detected:** Flags set to false. Alert ends after 5s timeout. LCD returns to telemetry cycle. Telemetry payload sends `"pir": false, "sound": false` .
- **GPS Handling:** TinyGPSPlus parses NMEA bytes from Serial2. If `gps.location.isValid()` is true, latitude/longitude are populated. If invalid, `gpsFix` sends `false` and coordinates send `null` . Server performs reverse geocoding via OpenStreetMap Nominatim API with fallback offline boundary mapping (e.g., mapping `12.8224, 77.5770` to "Buthanahalli (Bannerghatta NP Fringe, KA)").

5. CAMERA + AI ANALYSIS

- **Main Detection File:** `WelcomeScreen/fall_final/human_fall_detect_final_count.py` .
- **Model Used:** SSD MobileNet v3 Large COCO (`ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt.txt` and `frozen_inference_graph.pb`).
- **Model Type:** Object Detection Bounding Box Classifier.
- **Framework:** OpenCV DNN (`cv2.dnn_DetectionModel`).
- **Input Source / Camera Source:** OpenCV Video Capture (`cv2.VideoCapture(0, cv2.CAP_DSHOW)` at 1280x720 resolution).
- **Image Preprocessing:** Frame resized to 320x320, `scale = 1.0 / 127.5` , `mean = (127.5, 127.5, 127.5)` , `RGB swap = True` .
- **Detection Threshold:** Confidence threshold = `0.45` (45%), NMS threshold = `0.4` .
- **Elephant Class Handling:** Reads `coco.data` . Verifies "elephant" exists in class list. Iterates through detections and filters strictly for `class_name == "elephant"` .
- **Elephant Counting:** Count is computed dynamically as `len(elephant_detections)` .
- **Confidence Calculation:** `highest_confidence = max(confidence_values) * 100` .
- **Output Format:**
- Visual Bounding Boxes & Text overlay on OpenCV GUI window ("Elephant 1 - 94.5%").
- Local JPEG snapshot saved as `WelcomeScreen/fall_final/detected_elephants/elephant_YYYYMMDD_HHMMSS.jpg` .
- CSV dataset appended to `WelcomeScreen/fall_final/elephant_detection_dataset.csv` .
- Text-to-speech audio alert ("Warning! Elephant detected.") via `pyttsx3` .
- Asynchronous background thread `sync_detection_to_backend()` posts HTTP JSON to `http://127.0.0.1:5000/api/camera/detection-event` with fallback direct Supabase storage upload and database row insert.

Inspection of `human_fall_detect_final_count.py`

- **What it actually does:** Runs a continuous OpenCV camera loop, performing MobileNet SSD inferencing on live camera frames to count elephants, draw bounding boxes, log CSV data, save snapshot images, trigger audio voice warnings, and sync results to Supabase/Flask backend.
- **How it is started:** Can be run directly from terminal (`python human_fall_detect_final_count.py`) or programmatically spawned by `server.py` via `trigger_human_fall_detection()` .
- **What input it receives:** Direct video feed from local webcam (`index 0`).
- **What output it produces:** Local JPEG images, CSV rows, OpenCV display window, TTS audio speech, and HTTP API webhook payloads.
- **Continuous Execution:** Runs in a continuous `while True` loop until key `'q'` is pressed in the OpenCV window or the process is killed.
- **External Triggering:** Can be triggered externally by invoking POST `/api/detection/trigger` or when ESP32 PIR/Sound telemetry is ingested by `server.py` .
- **Function/File to trigger it:** Function `trigger_human_fall_detection(source)` in `server.py` which executes `subprocess.Popen([sys.executable, FALL_DETECT_SCRIPT])` .

6. SUPABASE ANALYSIS

- **Supabase URL Configuration:** Read from `.env` key `SUPABASE_URL` .
- **Client Initialization:** Handled in `supabase_client.py` using `supabase.create_client()` . Exposes administrative client (`get_supabase_admin_client()`) with `SUPABASE_SERVICE_ROLE_KEY` and public auth client (`get_auth_client()`) with `SUPABASE_PUBLISHABLE_KEY` .
- **Authentication Usage:** Supabase Auth is integrated for user login (`supabase.auth.sign_in_with_password`) and user management (`supabase.auth.admin.create_user` , `list_users` , `update_user_by_id`).
- **Storage Buckets:** `elephant-images` (Public/Private bucket for snapshot images).
- **Edge Functions:** None found in repository.

Supabase Table CRUD Matrix

Table	Used By	Read	Insert	Update	Delete
<code>devices</code>	<code>server.py</code> , <code>init_nodes_from_supabase()</code>	Yes	Yes (Upsert)	Yes	No
<code>detections</code>	<code>server.py</code> , <code>sync_detection_record()</code> , <code>camera_detection.js</code>	Yes	Yes	Yes	No
<code>raw_sensor_data</code>	<code>server.py</code> , <code>handle_esp32_data()</code> , <code>sensor_monitoring.js</code>	Yes	Yes	No	No
<code>alerts</code>	<code>server.py</code> , <code>notification_service.py</code> , <code>whatsapp_service.py</code>	Yes	Yes	Yes	No
<code>profiles</code>	<code>server.py</code> , <code>admin_create_user()</code> , <code>migration_v1_production.sql</code>	Yes	Yes	Yes	No
<code>system_settings</code>	<code>migration_v1_production.sql</code>	Yes	Yes	No	No
<code>notification_config</code>	<code>migration_v1_production.sql</code>	Yes	Yes	Yes	Yes
<code>notification_history</code>	<code>migration_v1_production.sql</code>	Yes	Yes	Yes	No

7. DATABASE SCHEMA

NOTE

> *Schema verified from SQL DDL files ('migration_v1_production.sql', 'schema_profiles.sql') and ORM calls in Python backend files.*

1. devices

- `device_id` (VARCHAR/TEXT, Primary Key): Unique device hardware identifier (e.g. `ESP32-NODE-01`).
- `name` (VARCHAR): Human-readable device name.
- `location_name` (VARCHAR): Reverse-geocoded location text.
- `latitude` (DOUBLE PRECISION): GPS latitude coordinate.
- `longitude` (DOUBLE PRECISION): GPS longitude coordinate.
- `status` (VARCHAR): Device status (`online` , `offline` , `maintenance`).
- `battery_level` (INTEGER): Battery percentage (e.g., `100`).
- `last_seen` (TIMESTAMPTZ): Timestamp of last received telemetry.
- `last_heartbeat_at` (TIMESTAMPTZ): Timestamp of last heartbeat signal.
- `firmware_version` (VARCHAR): Firmware version string.
- `created_at` / `updated_at` (TIMESTAMPTZ): Audit timestamps.

2. detections

- `id` (BIGSERIAL / UUID, Primary Key): Unique detection record ID.
- `device_id` (VARCHAR, Foreign Key -> `devices.device_id`): Node originating the detection.
- `timestamp` (TIMESTAMPTZ): Exact time of detection.
- `detection` (VARCHAR): Detection state (`ELEPHANT_DETECTED` or `SAFE`).
- `elephant_count` (INTEGER): Number of elephants counted by AI.
- `confidence` (NUMERIC/FLOAT): AI confidence normalized between `0.00` and `1.00` .
- `image_path` (VARCHAR): Filename of saved snapshot in `elephant-images` storage bucket.
- `pir_detected` (BOOLEAN): PIR sensor status during event.
- `sound_detected` (BOOLEAN): Sound sensor status during event.
- `sound_level` (NUMERIC): Acoustic decibel level (dB).
- `latitude` / `longitude` (DOUBLE PRECISION): Location coordinates of incident.
- `incident_status` (VARCHAR): Resolution status (`pending` , `acknowledged` , `responding` , `resolved`).
- `alert_sent` (BOOLEAN): Flag indicating if notifications were dispatched.

3. raw_sensor_data

- `id` (BIGSERIAL, Primary Key): Sensor reading entry ID.
- `device_id` (VARCHAR, Foreign Key -> `devices.device_id`): Originating hardware node.
- `timestamp` (TIMESTAMPTZ): Telemetry timestamp.
- `pir_detected` (BOOLEAN): Raw PIR state.
- `sound_detected` (BOOLEAN): Raw sound sensor state.
- `sound_level` (NUMERIC): Acoustic level in dB.
- `latitude` / `longitude` (DOUBLE PRECISION): Raw GPS coordinates.
- `created_at` (TIMESTAMPTZ): Ingestion timestamp.

4. alerts

- `id` (UUID / BIGSERIAL, Primary Key): Alert record ID.
- `detection_id` (VARCHAR/TEXT, Foreign Key -> `detections.id`): Associated detection event.
- `device_id` (VARCHAR, Foreign Key -> `devices.device_id`): Hardware node.
- `alert_type` (VARCHAR): Channel type (`WHATSAPP` , `WEB_PUSH` , `SMS`).
- `recipient` (VARCHAR): Recipient phone number or push endpoint.
- `message` (TEXT): Alert notification message body.
- `status` (VARCHAR): Delivery status (`sent` , `failed` , `active` , `pending`).
- `provider_message_id` (VARCHAR): External provider transaction ID (e.g. Baileys message ID).
- `error_message` (TEXT): Error traceback if delivery failed.
- `cooldown_triggered_at` (TIMESTAMPTZ): Cooldown timestamp.
- `created_at` / `sent_at` (TIMESTAMPTZ): Timestamps.

5. profiles

- `id` (UUID, Primary Key): Profile record ID.
- `user_id` (UUID, Foreign Key -> `auth.users.id` ON DELETE CASCADE): Linked Supabase Auth user ID.
- `full_name` (VARCHAR): User's full name.
- `email` (VARCHAR): User's email address.
- `phone` (VARCHAR): User's phone number.
- `role` (VARCHAR): Role identifier (`forest_officer` , `police` , `villager`).
- `location` (VARCHAR): User assigned zone or area.
- `status` (VARCHAR): Account status (`Active` or `Inactive`).
- `created_at` / `updated_at` (TIMESTAMPTZ): Timestamps.

6. system_settings

- `id` (BIGSERIAL, Primary Key).
- `setting_key` (VARCHAR, Unique): Configuration key (e.g., `NOTIFICATION_COOLDOWN_SECONDS`).
- `setting_value` (TEXT): Configuration value (e.g., `300`).
- `description` (TEXT): Setting explanation.

8. AUTHENTICATION + USER ROLES

- **Login System:** REST API `/api/auth/login` interacting with Supabase Authentication (`supabase.auth.sign_in_with_password`). Supports email login or phone/badge mapping.
- **Session Management:** Secure HTTP-only Flask server sessions (`session['user_id']` , `session['role']` , `session['name']` , `session['zone']`).
- **Role Access Matrix:**

Role	Access Path / Routes	Accessible Dashboard & Permissions
Forest Officer (forest_officer)	/officer , /dashboard , /detections , /raw-data , /elephant-images , /alerts , /users , /devices , /settings , /admin	Full Administrative Access: Comprehensive multi-tab monitoring dashboard (index.html), GIS map, live telemetry, camera stream, deterrent controls, alert management, and User Management (/api/admin/users).
Police (police)	/police , /api/police/*	Police Incident Dashboard: View active elephant incidents, threat confidence, affected road/village zones, and update dispatch status (Pending , Acknowledged , Responding , Resolved).
Villager (villager)	/villager , /api/villager/*	Villager Safety Dashboard: Simplified emergency safety alert card, distance from village boundary, safety guidelines, and audio warning popups.

- **Role-Based Access Control (RBAC):** Enforced via `@role_required(['role_name'])` python decorator in `server.py` . Non-officer attempts to access `/admin` or `/users` are blocked with HTTP 403 Forbidden (`403.html`).

9. DASHBOARD ANALYSIS

Routes List

- `/landing` : Public landing page with role choice cards.
- `/officer` (also `/dashboard` , `/detections` , `/raw-data` , `/elephant-images` , `/alerts` , `/users` , `/devices` , `/settings`): Forest Officer Master Dashboard.
- `/admin` (also `/user-management` , `/forest-officer/users`): Direct link to User Management tab in Master Dashboard.
- `/villager` : Villager Safety Portal.
- `/police` : Police Dispatch Portal.

Data Origin Classification (Real vs Static vs Mock)

[Dashboard Component]	[Data Type]	[Origin / Source]
1. GIS Corridor Map Nodes	REAL	Supabase `devices` & live ESP32 GPS
2. Telemetry Gauges (PIR/Sound/dB)	REAL	Supabase `raw_sensor_data` / ESP32 POST
3. Camera Detection Stream & Images	REAL	OpenCV camera + `detected_elephants/`
4. Detection History Table	REAL	Supabase `detections` table
5. User Management List	REAL	Supabase Auth Admin API & `profiles`
6. WhatsApp Notification Status	REAL	Local Baileys Service (Port 3001)
7. Edge-AI Audio Predict Simulator	MOCK	Simulated JSON in `/api/edge-ai/predict`
8. Decision Support System (DSS)	PARTIAL / HARDCODED	Dynamic node counts + static text strings
9. Strobe Light Simulator Overlay	REAL / SIMULATED	Toggles UI overlay & HTTP POST to ESP32

10. REAL-TIME DATA

- **Supabase Realtime:** Frontend JavaScript connects to Supabase Realtime WebSocket channel `schema-db-changes` listening to `INSERT` and `UPDATE` events on `detections` , `raw_sensor_data` , and `devices` .
- **API Polling:** As a fail-safe, `app.js` , `sensor_monitoring.js` , and `camera_detection.js` poll `/api/status` , `/api/nodes` , `/api/sensor/current` , and `/api/elephant-detection/confirmed` every 2 to 3 seconds.
- **Identified Mock / Hardcoded Fallbacks:**

- `/api/edge-ai/predict` : Returns randomized synthetic MFCC array vectors and pre-set text classifications (`Elephant Trumpet` , `Elephant Infrasonic Rumble` , `Environmental Wind Noise` , `Vehicle Motor Noise`).
- Offline device GPS fallback: If GPS fix is false (`gpsFix=false`), default coordinates `12.8224`, `77.5770` (Buthanahalli) are displayed with status `"LAST KNOWN LOCATION"` or `"NO LOCATION"` .

11. OFFLINE BEHAVIOR

- **Device Offline Detection:** In `server.py` , `check_device_connection()` compares `last_seen` timestamps against `ESP32_TIMEOUT_SECONDS = 30` . If no telemetry is received for > 30 seconds, status transitions to `OFFLINE` .
- **Live Sensor Readings:** `get_current_sensor_data()` returns `"pir_status": "OFFLINE"` , `"sound_status": "OFFLINE"` , and values reset to `0.0` .
- **Historical Data & Charts:** Historical time-series charts remain populated from Supabase `raw_sensor_data` records.
- **Detections & Images:** Past detections and saved images remain fully visible in the UI via Supabase Storage signed URLs and local disk fallbacks.
- **Notification Behavior:** If an ESP32 is offline, PIR/Sound triggers will not execute, preventing false alerts.

12. NOTIFICATION SYSTEM

- **Email:** Legacy fields exist in `system_settings` , but no active SMTP library or email sending service is present in code.
- **SMS:** Legacy configuration in `notification_service.py` (`_dispatch_twilio_sms`) redirects to local WhatsApp service.
- **WhatsApp:**
- **Trigger:** Invoked when `sync_detection_record()` processes a confirmed elephant detection (`detection="ELEPHANT_DETECTED"`).
- **Provider:** Local Baileys WhatsApp microservice (`whatsapp-service/server.js` running `@whiskeysockets/baileys` on port 3001).
- **Recipient:** Strictly restricted to authorized recipient `+917094528671` .
- **Message:** Constructed dynamically in `whatsapp_service.py` (`format_custom_elephant_message()`), containing elephant count, AI confidence %, location, timestamp, and GPS coordinates. Includes image snapshot if available.
- **Storage:** Recorded in Supabase `alerts` table with `status="sent"` or `"failed"` along with provider transaction ID.
- **Duplicate Prevention / Debouncing:** Dual-layer protection:
 1. Idempotency check on `detection_id` via `is_detection_already_alerted()` .
 2. Cooldown timer (`WHATSAPP_COOLDOWN_SECONDS = 300` / 5 minutes) enforced in memory and against Supabase `alerts` table.
- **Web Push Notifications:** Delivered via `push_service.py` using `pywebpush` with VAPID credentials to registered browser service workers (`service-worker.js`).

Keywords Search Results

- `TWILIO` : Referenced in `requirements.txt` and legacy comments in `migration_v1_production.sql` . Not used for active alerts.
- `Evolution API` : Legacy stub in `whatsapp_service.py` (`send_evolution_whatsapp`), aliased directly to `send_local_baileys_whatsapp` .
- `Baileys` / `whatsapp` : Primary active WhatsApp engine located in `whatsapp-service/server.js` and `whatsapp_service.py` .
- `SMTP` / `email` : Stubs only in database settings. No active code.

13. ENVIRONMENT VARIABLES

IMPORTANT

> All actual secret keys and passwords have been hidden in accordance with safety rules.

Variable	Used By	Purpose	Secret Status
SUPABASE_URL	supabase_client.py , server.py	Supabase Cloud project URL	PRESENT — VALUE HIDDEN
SUPABASE_PUBLISHABLE_KEY	supabase_client.py , server.py	Supabase anon/public API key	PRESENT — VALUE HIDDEN
SUPABASE_SERVICE_ROLE_KEY	supabase_client.py	Administrative Supabase service role key	PRESENT — VALUE HIDDEN
SECRET_KEY	server.py	Flask session cookie signing secret	PRESENT — VALUE HIDDEN
WHATSAPP_RECIPIENT	whatsapp_service.py , whatsapp-service/server.js	Authorized phone recipient (+917094528671)	PRESENT — VALUE HIDDEN
WHATSAPP_PORT	whatsapp-service/server.js	Express server port (3001)	PRESENT — VALUE HIDDEN
WHATSAPP_SERVICE_URL	whatsapp_service.py	Local Baileys service URL (http://localhost:3001)	PRESENT — VALUE HIDDEN
WHATSAPP_COOLDOWN_SECONDS	whatsapp_service.py	Cooldown period between WhatsApp alerts (300)	PRESENT — VALUE HIDDEN
VAPID_PUBLIC_KEY	push_service.py , server.py	Web Push VAPID public key	PRESENT — VALUE HIDDEN
VAPID_PRIVATE_KEY	push_service.py	Web Push VAPID private key	PRESENT — VALUE HIDDEN
VAPID_CLAIMS_SUB	push_service.py	Web Push subscriber mailto identifier	PRESENT — VALUE HIDDEN

14. API ENDPOINTS

Method	Endpoint	Purpose	Caller	Authentication
POST	/api/auth/login	Authenticate user with Supabase Auth	landing.html	Public
POST	/api/auth/logout	Clear Flask session & logout	Header / UI	Session
GET	/api/auth/current-user	Check active user session details	App load	Session
GET	/api/admin/users	List all users from Supabase Auth Admin	user_management.js	Forest Officer
POST	/api/admin/users/create	Provision new Forest Officer/Police/Villager	user_management.js	Forest Officer
POST	/api/admin/users/toggle-status	Activate or deactivate user account	user_management.js	Forest Officer
POST	/api/esp32/data	Hardware ESP32 telemetry ingestion	ESP32 esp32_node.ino	Hardware (Public)
POST	/api/node-event	Alternative hardware telemetry endpoint	ESP32 hardware	Hardware (Public)
GET	/api/status	System health, node online count & alert state	Dashboard header	Public
GET	/api/nodes	List live nodes and telemetry states	map.js , sensor_nodes.js	Public
POST	/api/node/update-location	Update live node GPS coordinates	UI / Field map	Session
GET	/api/camera/status	Camera connection & latest detection state	camera_detection.js	Public
GET	/api/detections	Fetch detection history	camera_detection.js	Public
POST	/api/camera/detection-event	Webhook for OpenCV camera script detections	human_fall_detect_final_count.py	Internal / Public
GET	/api/elephant-detection/confirmed	Poll latest fresh detection for popup modal	elephant_popup.js	Public
POST	/api/detection/trigger	Trigger camera detection script programmatically	server.py , Test UI	Session
GET	/api/detection/status	Check if detection process is running	camera_detection.js	Public

Method	Endpoint	Purpose	Caller	Authentication
POST	/api/deterrent/trigger	Trigger strobe light / siren / bio-acoustic	server.py , Dashboard	Forest Officer
GET	/api/sensor/current	Fetch live sensor telemetry for selected node	sensor_monitoring.js	Public
GET	/api/sensor/history	Fetch historical sensor telemetry time series	sensor_monitoring.js	Public
GET	/api/sensor/export	Export sensor readings as CSV file	sensor_monitoring.js	Public
GET	/api/villager/alerts	Fetch emergency alerts for villager view	villager_dashboard.html	Villager
GET	/api/police/incidents	Fetch incident list for police view	police_dashboard.html	Police
POST	/api/police/update-incident-status	Update incident status (Responding, Resolved)	police_dashboard.html	Police
POST	/api/push/subscribe	Register VAPID Web Push subscription	push_subscription.js	Session
POST	/api/alerts/test-whatsapp	Trigger test WhatsApp alert to recipient	Dashboard UI	Forest Officer
GET	/api/supabase/storage/signed-url	Generate signed URL for storage image	Frontend image load	Public

15. FILE-BY-FILE IMPORTANT COMPONENTS

1. server.py

- **Purpose:** Primary Flask web server, REST API engine, authentication gatekeeper, process manager for camera AI, and Supabase synchronization hub.
- **Called by:** WSGI / Gunicorn / direct execution (`python server.py`).
- **Calls:** `supabase_client` , `notification_service` , `whatsapp_service` , `push_service` , `human_fall_detect_final_count.py` .
- **Reads:** `.env` , Supabase tables (`devices` , `detections` , `raw_sensor_data` , `alerts`), local JSON files (`police_incidents.json`).
- **Writes:** Supabase tables (`raw_sensor_data` , `devices` , `detections` , `alerts`), Flask sessions, local JSON files.
- **Important Functions:** `trigger_human_fall_detection()` , `handle_esp32_data()` , `sync_detection_record()` , `start_csv_detection_watcher()` , `role_required()` .

2. WelcomeScreen/fall_final/human_fall_detect_final_count.py

- **Purpose:** Computer vision elephant detection engine using OpenCV DNN and MobileNet SSD.
- **Called by:** `server.py` via `trigger_human_fall_detection()` subprocess or direct command-line launch.
- **Calls:** OpenCV DNN API, `pyttsx3` , requests (HTTP POST to `/api/camera/detection-event`) , `supabase` Python SDK.

- **Reads:** `coco.data` , `ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt.txt` , `frozen_inference_graph.pb` , webcam stream.
- **Writes:** `elephant_detection_dataset.csv` , JPEG snapshots to `detected_elephants/` , audio speech via speakers, Supabase storage and database.

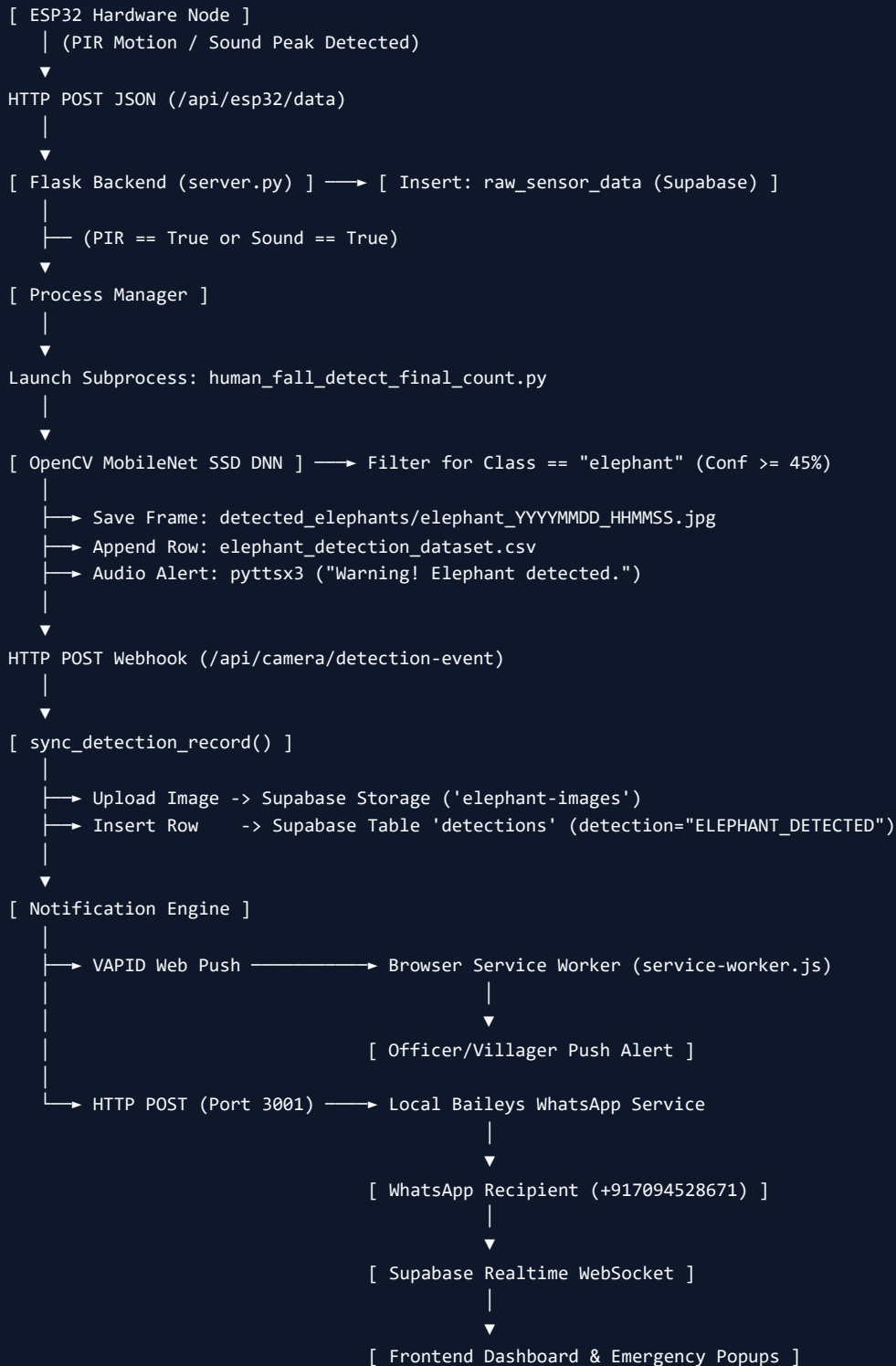
3. `whatsapp-service/server.js`

- **Purpose:** Standalone local Node.js microservice maintaining persistent WhatsApp Web connection via `@whiskeysockets/baileys` .
- **Called by:** `whatsapp_service.py` via HTTP POST requests to `http://localhost:3001` .
- **Calls:** Baileys WASocket protocol handler, local file system for session credentials.
- **Reads:** `.env` , `sessions/` multi-file auth state.
- **Writes:** WhatsApp protocol sockets, `sessions/` auth credentials, QR code data URL.

4. `hardware_integration/esp32_node.ino`

- **Purpose:** Firmware for ESP32 field hardware node. Reads PIR, sound, and GPS sensors, updates LCD display, and posts HTTP JSON telemetry.
 - **Called by:** ESP32 Microcontroller hardware loop.
 - **Calls:** HTTPClient to Flask server `/api/esp32/data` .
 - **Reads:** PIR Pin D19, Sound Pin D18, GPS Serial2 Pins D16/D17, I2C LCD.
 - **Writes:** Relay Pin D23 (strobe light), Status LED Pin D22, I2C LCD screen, HTTP POST requests.
-

16. DATA FLOW DIAGRAM



17. CURRENT PROBLEMS / RISKS

CRITICAL

1. **Hardcoded IP Address in ESP32 Firmware:** `esp32_node.ino` line 127 hardcodes `SERVER_URL = "http://10.238.159.122:5000/api/esp32/data"`. If the host server IP changes on local Wi-Fi, telemetry ingestion breaks completely.

2. **Webcam Conflict / Camera Locking:** `human_fall_detect_final_count.py` acquires camera `0` (`cv2.VideoCapture(0, cv2.CAP_DSHOW)`). Simultaneously, Flask `server.py` defines `/api/camera/stream` which also attempts to open camera `0`. Concurrent access to camera `0` on Windows causes device lock errors or silent frame drop failures.

HIGH

1. **Single Authorized WhatsApp Recipient Hardcoding:** `whatsapp-service/server.js` and `whatsapp_service.py` strictly restrict WhatsApp messaging to `+917094528671`. Alerts to actual villagers or dynamic phone numbers are blocked by design during testing mode.
2. **Subprocess Spawn Overhead on Windows:** Launching `human_fall_detect_final_count.py` via `subprocess.Popen` creates a new Python interpreter process and reloads heavy OpenCV DNN weights (~13.4 MB `.pb` file) on every new trigger event, causing a 2-4 second initialization latency.

MEDIUM

1. **In-Memory Telemetry Data Race:** `NODES_DATA` is modified globally across request threads without explicit threading lock synchronization.
2. **Mixed Database Operations:** Several endpoints fall back to local disk JSON files (`police_incidents.json`, `alert_history.json`) when Supabase connectivity encounters latency.

LOW

1. **Mock Data in Edge-AI Predict API:** `/api/edge-ai/predict` returns static simulated responses for audio classification rather than inferencing live audio WAV files.

18. CURRENT WORKING FEATURES

-  **ESP32 Sensor Ingestion:** Working HTTP POST telemetry from ESP32 to `/api/esp32/data`.
-  **Supabase Database Storage:** Real-time persistence for sensor readings, devices, and detections.
-  **Camera Elephant Detection:** OpenCV MobileNet SSD bounding box classification and count calculation.
-  **Local WhatsApp Delivery:** Working WhatsApp alert dispatch via local Baileys microservice to `+917094528671`.
-  **Web Push Notifications:** VAPID Web Push delivery to browser service worker.
-  **Multi-Role Dashboards:** Working role-based routing and permissions for Forest Officer, Police, and Villager portals.
-  **Supabase Auth Integration:** User login and role provisioning via Supabase Auth Admin APIs.
-  **GPS Reverse Geocoding:** Partially dependent on Nominatim OpenStreetMap API rate limits (mitigated by offline regional fallbacks).
-  **Deterrent Relay Actuation:** Working manual toggle over HTTP, but requires static IP mapping to ESP32 node.
-  **Twilio SMS / Email Delivery:** Replaced by WhatsApp; active SMS/Email pipelines not implemented.

19. WHAT SHOULD NOT BE CHANGED

1. **Working ESP32 Communication Protocol:** HTTP JSON POST schema targeting `/api/esp32/data` (`deviceId`, `pir`, `sound`, `gpsFix`, `latitude`, `longitude`).
2. **MobileNet SSD Model Files & Weights:** `frozen_inference_graph.pb`, `ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt.txt`, and `coco.data` structure.
3. **Supabase Schema Core Tables:** `devices`, `detections`, `raw_sensor_data`, `alerts`, and `profiles` table definitions and RLS policies.

4. **Local Baileys WhatsApp Service Architecture:** whatsapp-service/server.js multi-file auth flow and REST endpoints (/send-image , /send-message).
5. **Two-Stage Detection State Machine Concept:** Sensor trigger -> Camera verification -> Confirmed alert dispatch.

20. FINAL PROJECT SUMMARY

PROJECT:

Integrated IoT & Edge-AI Acoustic Early Warning System for Human-Elephant Conflict Mitigation

PURPOSE:

Real-time early detection of wild elephants along forest fringes (e.g. Bannerghatta / Anekal corridor) using IoT field sensors (PIR + Sound), Edge-AI computer vision (MobileNet SSD), and multi-channel notifications (WhatsApp, Web Push) to prevent human-animal conflict.

FRONTEND:

HTML5, Custom CSS Design System, Vanilla JS (ES6), Jinja2 templates (`index.html`, `landing.html`, `villager\_dashboard.html`, `police\_dashboard.html`), Leaflet.js GIS maps, Chart.js telemetry graphs.

BACKEND:

Python Flask framework (`server.py`) serving REST APIs + Node.js Express microservice (`whatsapp-service/server.js`) for Baileys WhatsApp integration.

AI:

OpenCV DNN module running MobileNet SSD v3 Large trained on COCO dataset, filtered for class `elephant` with confidence threshold $\geq 45\%$.

IOT:

ESP32 DevModule with PIR motion sensor (Pin 19), Sound sensor (Pin 18), TinyGPSPlus GPS module (UART2), 16x2 I2C LCD, and Strobe/Siren Relay (Pin 23). Communicates over Wi-Fi HTTP POST.

DATABASE:

Supabase PostgreSQL Cloud (`devices`, `detections`, `raw\_sensor\_data`, `alerts`, `profiles`) + Supabase Storage (`elephant-images` bucket).

AUTH:

Supabase Auth with Role-Based Access Control (Forest Officer, Police, Villager).

NOTIFICATIONS:

Local Baileys Node.js WhatsApp service (delivering alerts & images to +917094528671) + VAPID Web Push notifications + local TTS voice alerts (`pyttsx3`).

CURRENT DETECTION FLOW:

ESP32 Sensors (PIR/Sound) -> Flask Server Ingestion -> Subprocess Spawn of `human\_fall\_detect\_final\_count.py` -> OpenCV Camera Frame Analysis -> Image Saved & CSV Logged -> Supabase Storage & DB Insert -> Local Baileys WhatsApp & Web Push Alert -> Dashboard Live Map & Popup Update.

CURRENT WHATSAPP FLOW:

`sync\_detection\_record()` -> `whatsapp\_service.send\_confirmed\_elephant\_whatsapp()` -> HTTP POST to `http://localhost:3001/send-image` -> Baileys WASocket -> Delivery to +917094528671 -> Record logged in Supabase `alerts` table.

CURRENT MAIN ENTRY POINTS:

- Main Web Server: `python Integrated-IoT-and-Edge-AI-Acoustic-Early-Warning-System-for-Human-Elephant-Conflict-Mitigation-main/server.py`
- WhatsApp Service: `node whatsapp-service/server.js`
- Camera Detection Script: `python WelcomeScreen/fall\_final/human\_fall\_detect\_final\_count.py`
- Hardware Firmware: `Integrated-IoT-and-Edge-AI-Acoustic-Early-Warning-System-for-Human-Elephant-Conflict-Mitigation-main/hardware\_integration/esp32\_node.ino`